# Trust Boundaries on Registry Trees: Duplicate-Stability of the Boundary Rule

Extracted from a machine-checked development

**Abstract**

We treat trust-labelled, finitely branching rose trees as a directed container and formalise the *boundary rule* used to validate proof registries: a node may claim a threshold trust level only if every non-dead-end child claims at least that level. The global rule is the coKleisli form of the local check—the local predicate holds at every position of the tree. We prove *duplicate-stability*: if a tree satisfies the global boundary rule, then every subtree does. Labels do not disturb the navigation laws of the underlying directed container. Two concrete trust chains (a seven-level agent chain and a six-level loop-experiment chain) instantiate the general result. All stated results have been machine-checked.

## Contents

**1 Introduction** **1**

**2 Trust labels and labelled trees** **2**

**3 Positions, subtrees, and path concatenation** **2**

**4 The boundary rule** **3**

**5 Duplicate-stability** **3**

**6 Deployed trust chains** **4**

**7 Discussion** **4**

## 1 Introduction

Proof registries may be organised as finitely branching trees whose nodes carry trust labels. A validator walks the tree and enforces a node-local rule: a claim graded at or above a threshold may stand only if every live child is graded at or above that threshold. Dead ends (abandoned approaches) are exempt. Shared nodes import subtrees across registry links; for validation to be compositional, a validated tree must contribute only validated subtrees.

Directed containers supply the navigation structure: shapes are trees, positions are paths from the root, and the comonad operations extract and duplicate arise from path concatenation. The boundary rule, stated at every position produced by duplicate, is stable under taking subtrees—transport along the directed-container law that subtrees nest along path concatenation.

**What this note claims.** In ordinary mathematical language: validating the whole tree validates every subtree. More precisely, if the local boundary check holds at every position of a labelled tree $t$, then it holds at every position of every subtree $t \downarrow p$. The proof is a one-line

transport along the nesting law for subtrees. Dead ends are modelled by a boolean flag outside the trust order, because an order relation cannot express an exemption. Two deployed trust chains instantiate the abstract threshold.

## 2 Trust labels and labelled trees

**Definition 2.1** (Trust label)**.** Let $T$ be a type of trust levels. A **trust label** is a pair

$$\ell \;=\; \langle \mathrm{trust}(\ell),\ \mathrm{dead}(\ell) \rangle \;\in\; T \times \mathbb{B}, \tag{1}$$

consisting of a level in $T$ and a dead-end flag. Dead ends lie outside the trust order by design: the boundary rule *exempts* them, and no order relation expresses an exemption.

**Definition 2.2** (Labelled rose tree)**.** A **trust-labelled rose tree** over $T$ is a finitely branching tree with a label at every node:

$$t \;=\; \mathrm{node}(\ell,\ n,\ (t_i)_{i<n}), \tag{2}$$

where $\ell$ is a trust label, $n \in \mathbb{N}$ is the branching factor, and each $t_i$ is a labelled rose tree. A leaf is a node with $n = 0$.

**Definition 2.3** (Root label)**.** The **root label** of $t = \mathrm{node}(\ell, n, (t_i))$ is $\mathrm{lab}(t) = \ell$. Under the comonad structure below, $\mathrm{lab}(t \downarrow p)$ reads the label at position $p$.

## 3 Positions, subtrees, and path concatenation

**Definition 3.1** (Position)**.** A **position** (path) in a labelled tree $t$ is generated by

- $\mathsf{here}$ — the root of $t$;
- $\mathsf{child}(i, p)$ — descend into the $i$-th child and continue along path $p$.

For a leaf ($n = 0$), the only position is $\mathsf{here}$.

**Definition 3.2** (Subtree and shift)**.** For a tree $t$ and position $p$ in $t$:

- the **subtree** $t \downarrow p$ is $t$ itself at $\mathsf{here}$, and the recursively computed subtree of the $i$-th child at $\mathsf{child}(i, p)$;
- the **shift** (path concatenation) $p \oplus q$ embeds a position $q$ in the subtree at $p$ back into $t$: at $\mathsf{here}$ it is $q$, and at $\mathsf{child}(i, p)$ it is $\mathsf{child}(i,\ p \oplus q)$.

**Proposition 3.3** (Directed-container laws)**.** *Writing $o(t) = \mathsf{here}$ for the root position, the following hold for all trees and positions (labels play no role):*

***D1.*** $t \downarrow o(t) \;=\; t$.

***D2.*** $o(t) \oplus q \;=\; q$.

***D3.*** $p \oplus o(t \downarrow p) \;=\; p$.

***D4.*** $t \downarrow (p \oplus q) \;=\; (t \downarrow p) \downarrow q$.

***D5.*** $(p \oplus q) \oplus r \;=\; p \oplus (q \oplus r)$ *(up to the identification of fibres supplied by D4).*

*In particular, labelled trees form a directed container; the navigation structure—and hence the induced comonad—is independent of the labels.*

*Proof.* By lean. □

**Remark 3.4** (Comonad reading)**.** The induced comonad on the extension has

$$\begin{aligned} \text{extract}\,\langle t, v\rangle &= v(\mathsf{here}), && (3)\\ \text{duplicate}\,\langle t, v\rangle &= \langle t,\ p \mapsto \langle t \downarrow p,\ q \mapsto v(p \oplus q)\rangle\rangle. && (4) \end{aligned}$$

At every node, duplicate replaces the label with the full sub-search from that point.

# 4 The boundary rule

We now assume $T$ carries a partial order $\leq$ (in applications, a linear chain of check strengths with bottom element).

**Definition 4.1** (Local boundary rule)**.** Fix a threshold $\theta \in T$. The **local boundary rule** at a tree $t = \text{node}(\ell, n, (t_i))$ is the proposition

$$\text{localOk}_\theta(t) \iff \Big(\theta \leq \text{trust}(\ell) \implies \forall\, i < n.\ \big(\text{dead}(\text{lab}(t_i)) \ \vee\ \theta \leq \text{trust}(\text{lab}(t_i))\big)\Big). \tag{5}$$

If the node claims $\theta$ or above, every non-dead child claims $\theta$ or above. There is no recursion: the check inspects one node and its immediate children.

**Definition 4.2** (Global boundary rule)**.** The **global boundary rule** at threshold $\theta$ is the coKleisli form of the local check: the local rule holds at *every* position,

$$\text{Ok}_\theta(t) \iff \forall\, p \text{ a position in } t.\ \text{localOk}_\theta(t \downarrow p). \tag{6}$$

Equivalently: the local predicate holds on every sub-search produced by duplicate.

# 5 Duplicate-stability

**Theorem 5.1** (Duplicate-stability of the trust boundary)**.** *Let $\theta \in T$ and let $t$ be a labelled tree. If $\text{Ok}_\theta(t)$, then for every position $p$ in $t$,*

$$\text{Ok}_\theta(t \downarrow p). \tag{7}$$

*Proof.* By lean. □

**Remark 5.2** (Idea of the argument)**.** Positions of the subtree $t \downarrow p$ embed into positions of $t$ by path concatenation $q \mapsto p \oplus q$. The nesting law D4 identifies

$$(t \downarrow p) \downarrow q \ = \ t \downarrow (p \oplus q),$$

so the global check at $p \oplus q$ *is* the check at $q$ in the subtree. Transport along this equality yields the result.

**Theorem 5.3** (Whole equals all duplicates)**.** *For every threshold $\theta$ and tree $t$,*

$$\text{Ok}_\theta(t) \iff \forall\, p \text{ a position in } t.\ \text{Ok}_\theta(t \downarrow p). \tag{8}$$

*The forward direction is Theorem 5.1; the backward direction reads the root component of each duplicate.*

*Proof.* By lean. □

**Remark 5.4.** This is the load-bearing fact for shared nodes: a stub pointing into a validated registry inherits a validated subtree, and the validator never needs to re-check across the link.

# 6 Deployed trust chains

**Definition 6.1** (Agent trust chain)**.** The **agent trust chain** is the seven-element linear order

$$\begin{aligned}\text{unchecked} \;<\; \text{speculative} \;<\; \text{computed} \;<\; \text{proved}\\ <\; \text{peerReviewed} \;<\; \text{published} \;<\; \text{leanVerified},\end{aligned}$$

with bottom element unchecked. The dead-end status is not a chain element: it is the dead flag on labels.

**Definition 6.2** (Registry validity)**.** A labelled tree over the agent trust chain is **registry-valid** if it satisfies the global boundary rule at threshold proved:

$$\text{RegistryOk}(t) \iff \text{Ok}_{\text{proved}}(t). \tag{9}$$

**Corollary 6.3** (Deployed duplicate-stability)**.** *If $t$ is registry-valid, then every subtree of $t$ is registry-valid.*

*Proof.* By lean. □

**Definition 6.4** (Loop-experiment trust chain)**.** The **loop-experiment trust chain** is the six-element linear order

$$\begin{aligned}\text{unchecked} \;<\; \text{speculative} \;<\; \text{computed} \;<\; \text{proved}\\ <\; \text{verified} \;<\; \text{leanVerified},\end{aligned}$$

with bottom element unchecked. The grade verified replaces the social-process grades peerReviewed and published of Definition 6.1: it records an independent hostile review.

**Corollary 6.5** (Loop duplicate-stability)**.** *A labelled tree over the loop-experiment chain that satisfies* $\text{Ok}_{\text{proved}}$ *has every subtree satisfying* $\text{Ok}_{\text{proved}}$*.*

*Proof.* By lean. □

**Remark 6.6.** Both corollaries are instances of Theorem 5.1, which is parametric in the trust type $T$ and the threshold $\theta$. The same structural fact applies to any chain and any threshold.

# 7 Discussion

**What has been established.** Trust-labelled rose trees form a directed container whose navigation is independent of labels. The boundary rule—local at each node, global as a coKleisli condition—is stable under duplicate: validated trees decompose into validated subtrees. Two concrete chains used in deployed registries instantiate the general theorem.

**Relation to transport of trust.** Duplicate-stability says validated trees decompose. Separately, morphisms of trust systems transport canonical trust laxly: importing along a morphism never claims more than the image of the source's canonical trust. Together these are the two halves of compositional registry validation: structure decomposes, and grades move only downward under import.

**Machine checking.** Every definition and theorem statement above corresponds to a declaration in a machine-checked formalisation. The phrase "by lean" indicates that the proof has been verified by a proof assistant.

## Acknowledgements

The presentation here is a prose rendering of a formal development intended for human review of the theorem statements. The directed container laws follow the theory of Ahman, Chapman, and Uustalu; the boundary rule is the mathematical content of a node-local registry validator.

## References

[1] D. Ahman, J. Chapman, and T. Uustalu. When is a container a comonad? In *FoSSaCS*, volume 7213 of *LNCS*, pages 74–88. Springer, 2012.

[2] D. Ahman and T. Uustalu. Directed containers as categories. *EPTCS*, 207:89–98, 2016.